

Guías de Implementación

Mecanismos de Comunicación
en Sistemas Distribuidos con Java

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 1 Generalidades de los Sistemas Distribuidos

Nivel: Séptimo semestre Ingeniería de Software

IDE: IntelliJ IDEA 2024.x

Lenguaje: Java 21 (LTS) + Maven 3.9+

Período: Regular 2026-2027

Docente: Ing. Gleiston Guerrero Ulloa, Mg.Sc.

Contenido de este documento

1. **Guía 1** Sockets TCP y UDP
2. **Guía 2** WebSockets con Jakarta EE / Spring Boot
3. **Guía 3** RPC con Apache Thrift
4. **Guía 4** gRPC con Protocol Buffers
5. **Guía 5** RMI (Remote Method Invocation)

Cada guía incluye: fundamento teórico, diagrama de arquitectura, código fuente completo paso a paso, ejecución desde IntelliJ IDEA y verificación de resultados.

Índice general

Introducción	5
1. Guía 1: Sockets TCP y UDP	6
1.1. Fundamento teórico	6
1.1.1. TCP vs. UDP	6
1.2. Estructura del proyecto Maven	7
1.3. Implementación de Sockets TCP	8
1.4. Implementación de Sockets UDP	11
1.5. Ejecución desde IntelliJ IDEA	14
2. Guía 2: WebSockets con Spring Boot	16
2.1. Fundamento teórico	16
2.2. Crear proyecto Spring Boot	17
2.3. Configuración del servidor WebSocket	17
2.4. Ejecución	22
3. Guía 3: RPC con Apache Thrift	23
3.1. Fundamento teórico	23
3.2. Configuración del proyecto	23
3.3. Definición del IDL	24
3.4. Implementación servidor y cliente	26
3.5. Ejecución	29
4. Guía 4: gRPC con Protocol Buffers	30
4.1. Fundamento teórico	30
4.2. Configuración del proyecto	31
4.3. Definición del contrato Protobuf	32
4.4. Implementación del servidor gRPC	34
4.5. Ejecución	39
5. Guía 5: Java RMI (Remote Method Invocation)	40
5.1. Fundamento teórico	40

5.2. Estructura del proyecto	41
5.3. Interfaz remota	41
5.4. Implementación del objeto remoto	42
5.5. Cliente RMI	46
5.6. Ejecución	47
6. Análisis Comparativo	49
6.1. Cuadro sinóptico de mecanismos	49
6.2. Criterios de selección	49

Índice de figuras

1.1. Comparativa del flujo de establecimiento de conexión TCP vs. UDP . .	7
2.1. Ciclo de vida de una conexión WebSocket sobre HTTP	16
5.1. Arquitectura completa del sistema RMI con tres JVMs	41

Introducción

Los sistemas distribuidos modernos utilizan múltiples mecanismos de comunicación entre procesos (*Inter-Process Communication*, IPC). La elección del mecanismo correcto determina la latencia, el throughput, la complejidad del código y la interoperabilidad del sistema [vanSteen2023].

Este documento presenta cinco guías de implementación progresivas para Java 21 con IntelliJ IDEA, cubriendo el espectro completo desde los sockets de bajo nivel hasta la invocación de métodos remotos de alto nivel:

Cuadro 1: Comparativa de mecanismos de comunicación cubiertos en las guías

Mecanismo	Nivel	Protocolo	Tipado	Uso típico
Sockets TCP	Bajo	TCP/IP	Sin tipado	Streaming, chat
Sockets UDP	Bajo	UDP/IP	Sin tipado	VoIP, gaming, DNS
WebSocket	Medio	HTTP/WS	Sin tipado	Tiempo real, notif.
RPC (Thrift)	Alto	TCP/bin	Fuerte	Microservicios
gRPC	Alto	HTTP/2	Fuerte	Microservicios
RMI	Alto	JRMP	Java	Sistemas Java

Prerrequisitos

Antes de comenzar cualquier guía, verificar la instalación de:

- **JDK 21** (Adoptium Temurin 21 LTS): `java --version`
- **Maven 3.9+**: `mvn --version`
- **IntelliJ IDEA 2024.x** (Community o Ultimate)
- **Git 2.x**: para control de versiones del código

Capítulo 1

Guía 1: Sockets TCP y UDP

Objetivo: Sockets TCP y UDP

Implementar un sistema de mensajería distribuida usando sockets TCP (orientado a conexión, entrega garantizada) y UDP (sin conexión, baja latencia), comprendiendo las diferencias arquitectónicas entre ambos protocolos [stevens2004].

1.1. Fundamento teórico

Un **socket** es la abstracción del sistema operativo que permite a dos procesos en red intercambiar datos. Representa un endpoint de comunicación identificado por la tupla (IP, puerto, protocolo) [coulouris2012].

1.1.1. TCP vs. UDP

TCP (*Transmission Control Protocol*) establece una conexión antes de transferir datos mediante el *three-way handshake* ($\text{SYN} \rightarrow \text{SYN-ACK} \rightarrow \text{ACK}$). Garantiza: entrega en orden, detección y retransmisión de pérdidas, y control de flujo mediante ventana deslizante [kurose2017].

UDP (*User Datagram Protocol*) envía datagramas independientes sin establecer conexión previa. Ofrece latencia mínima al costo de no garantizar orden ni entrega, siendo adecuado cuando la pérdida ocasional de paquetes es tolerable (VoIP, juegos en línea, DNS) [kurose2017].

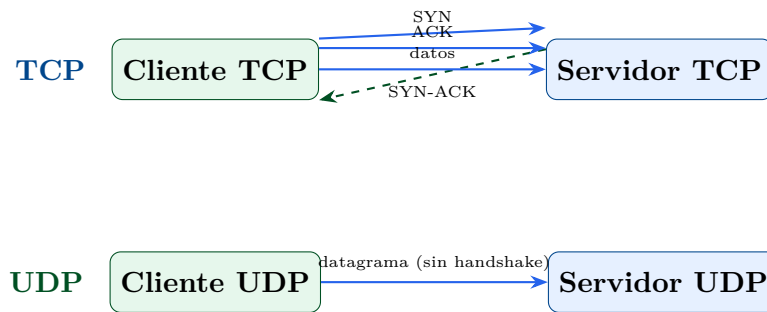


Figura 1.1: Comparativa del flujo de establecimiento de conexión TCP vs. UDP

1.2. Estructura del proyecto Maven

Paso 1. Abrir IntelliJ IDEA. Ir a *File* → *New* → *Project*. Seleccionar **Maven Archetype**, elegir `org.apache.maven.archetypes:maven-archetype-quickstart`, definir:

- **GroupId:** `ec.edu.uteq.distribuidas`
- **ArtifactId:** `guia01-sockets`
- **Version:** `1.0-SNAPSHOT`

Paso 2. Reemplazar el contenido de `pom.xml` con la siguiente configuración para Java 21:

```

1      <?xml version="1.0" encoding="UTF-8"?>
2      <project xmlns="http://maven.apache.org/POM/4.0.0"
3              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4              xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5      http://maven.apache.org/xsd/maven-4.0.0.xsd">
6          <modelVersion>4.0.0</modelVersion>
7          <groupId>ec.edu.uteq.distribuidas</groupId>
8          <artifactId>guia01-sockets</artifactId>
9          <version>1.0-SNAPSHOT</version>
10         <properties>
11             <maven.compiler.source>21</maven.compiler.source>
12             <maven.compiler.target>21</maven.compiler.target>
13             <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14         </properties>
15     </project>

```

Listing 1.1: `pom.xml` — Configuración del proyecto

1.3. Implementación de Sockets TCP

Paso 1. Crear el paquete `ec.edu.uteq.distribuidas.tcp` dentro de `src/main/java`.
Clic derecho sobre `ec.edu.uteq.distribuidas` → *New* → *Package* → ingresar `tcp`.

Paso 2. Crear la clase `ServidorTCP.java` en el paquete `tcp`:

```

1      package ec.edu.uteq.distribuidas.tcp;
2
3      import java.io.*;
4      import java.net.*;
5      import java.time.LocalDateTime;
6      import java.time.format.DateTimeFormatter;
7      import java.util.concurrent.*;
8      import java.util.concurrent.atomic.*;
9
10     /**
11     * Servidor TCP multihilo con pool de hilos fijo.
12     * Escucha en el puerto 9000 y atiende comandos:
13     *   HORA    -> retorna la hora actual del servidor
14     *   ECO     -> retorna el mensaje prefijado con ECO:
15     *   SALIR   -> cierra la conexion del cliente
16     */
17     public class ServidorTCP {
18
19         private static final int PUERTO = 9000;
20         private static final int MAX_HILOS = 10;
21         private static final AtomicInteger contadorClientes = new
22             AtomicInteger(0);
23
24         public static void main(String[] args) throws IOException
25         {
26             ExecutorService pool = Executors.newFixedThreadPool(
27                 MAX_HILOS);
28
29             try (ServerSocket servidor = new ServerSocket(PUERTO))
30             {
31                 System.out.println "[" + timestamp() + "] Servidor
32                     listo en : " + PUERTO);
33
34                 while (true) {
35                     Socket cliente = servidor.accept();    //
36                         BLOQUEA hasta nueva conexion
37                     int id = contadorClientes.incrementAndGet();
38                     System.out.println "[" + timestamp() + "]

```

```

33         Cliente #" + id
34         + " conectado desde " + cliente.getInetAddress
35         ());
36         pool.submit(new ManejadorCliente(cliente, id))
37         ;
38     }
39 }
40
41 public static String timestamp() {
42     return LocalDateTime.now()
43     .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
44 }
45
46 // Clase interna para manejar cada cliente en su propio
47 // hilo
48
49 static class ManejadorCliente implements Runnable {
50     private final Socket socket;
51     private final int idCliente;
52
53     ManejadorCliente(Socket socket, int idCliente) {
54         this.socket = socket;
55         this.idCliente = idCliente;
56     }
57
58     @Override
59     public void run() {
60         try (BufferedReader entrada = new BufferedReader(
61             new InputStreamReader(socket.getInputStream()));
62             PrintWriter salida = new PrintWriter(
63                 socket.getOutputStream(), true)) {
64
65             String linea;
66             while ((linea = entrada.readLine()) != null) {
67                 System.out.println("[ " + ServidorTCP.
68                     timestamp()
69                     + " ] Cliente#" + idCliente + " > " + linea
70                     );
71
72                 if (linea.equalsIgnoreCase("HORA")) {
73                     salida.println("HORA_ACTUAL:" +
74                         ServidorTCP.timestamp());
75                 } else if (linea.toUpperCase().startsWith(
76                     "ECO")) {
77                     String mensaje = linea.length() > 4
78                         ? linea.substring(4) : "(vacío)";

```

```

72         salida.println("ECO[#" + idCliente + "
73             ":" + mensaje);
74     } else if (linea.equalsIgnoreCase("SALIR")
75     ) {
76         salida.println("ADIOS:hasta luego
77             cliente #" + idCliente);
78         System.out.println "[" + ServidorTCP.
79             timestamp()
80             + "]" + " Cliente #" + idCliente + "
81             desconectado.");
82         break;
83     } else {
84         salida.println("ERROR:comando
85             desconocido -> " + linea);
86     }
87 }
88 }
89 }

```

Listing 1.2: ServidorTCP.java — Servidor multihilo con ThreadPool

Paso 3. Crear la clase **ClienteTCP.java** en el mismo paquete tcp:

```

1  package ec.edu.uteq.distribuidas.tcp;
2
3  import java.io.*;
4  import java.net.*;
5  import java.util.Scanner;
6
7  /**
8   * Cliente TCP interactivo.
9   * Envía comandos al servidor y muestra la respuesta.
10  * Uso: ejecutar desde IntelliJ -> Run 'ClienteTCP.main()'
11  */
12  public class ClienteTCP {
13
14      private static final String HOST = "localhost";
15      private static final int PUERTO = 9000;
16

```

```

17         public static void main(String[] args) throws IOException
18         {
19             try (Socket socket = new Socket(HOST, PUERTO);
20                 BufferedReader entrada = new BufferedReader(
21                     new InputStreamReader(socket.getInputStream()));
22                 PrintWriter salida = new PrintWriter(
23                     socket.getOutputStream(), true);
24                 Scanner teclado = new Scanner(System.in)) {
25
26                 System.out.println("Conectado a " + HOST + ":" +
27                     PUERTO);
28                 System.out.println("Comandos disponibles: HORA |
29                     ECO <texto> | SALIR");
30                 System.out.println("
31                     -----
32                     ");
33
34                 String comando;
35                 while (true) {
36                     System.out.print("> ");
37                     comando = teclado.nextLine().trim();
38                     if (comando.isEmpty()) continue;
39
40                     salida.println(comando);    // enviar al
41                                             servidor
42                     String respuesta = entrada.readLine();
43                     System.out.println("Servidor: " + respuesta);
44
45                     if (comando.equalsIgnoreCase("SALIR")) break;
46                 }
47             }
48             System.out.println("Conexion cerrada.");
49         }
50     }

```

Listing 1.3: ClienteTCP.java — Cliente interactivo

1.4. Implementación de Sockets UDP

Paso 1. Crear el paquete `ec.edu.uteq.distribuidas.udp` y la clase `ServidorUDP.java`:

```

1     package ec.edu.uteq.distribuidas.udp;
2

```

```

3      import java.net.*;
4      import java.time.LocalDateTime;
5      import java.time.format.DateTimeFormatter;
6
7      /**
8       * Servidor UDP sin conexion.
9       * Recibe datagramas de hasta 1024 bytes y responde en el
10      * mismo socket. No tiene estado entre paquetes.
11      */
12      public class ServidorUDP {
13
14          private static final int PUERTO    = 9001;
15          private static final int TAM_BUF   = 1024;
16
17          public static void main(String[] args) throws Exception {
18              try (DatagramSocket socket = new DatagramSocket(PUERTO)
19              ) {
20                  System.out.println("Servidor UDP escuchando en
21                  puerto " + PUERTO);
22
23                  byte[] buffer = new byte[TAM_BUF];
24                  while (true) {
25                      DatagramPacket paquete =
26                          new DatagramPacket(buffer, buffer.length);
27                      socket.receive(paquete);    // BLOQUEA hasta
28                                                  recibir datagrama
29
30                      String mensaje = new String(paquete.getData(),
31                      0, paquete.getLength());
32                      String hora = LocalDateTime.now()
33                      .format(DateTimeFormatter.ofPattern("HH:mm:ss.
34                      SSS"));
35
36                      System.out.println "[" + hora + "] Recibido de
37                      "
38                      + paquete.getAddress() + ":" + paquete.getPort()
39                      + " -> " + mensaje);
40
41                      // Preparar respuesta
42                      String respuesta = "ACK:" + mensaje + " @" +
43                      hora;
44                      byte[] resp = respuesta.getBytes();
45                      DatagramPacket respPkt = new DatagramPacket(
46                      resp, resp.length,
47                      paquete.getAddress(), paquete.getPort());
48                      socket.send(respPkt);

```

```

43         }
44     }
45 }
46

```

Listing 1.4: ServidorUDP.java — Servidor sin conexión

```

1  package ec.edu.uteq.distribuidas.udp;
2
3  import java.net.*;
4  import java.util.Scanner;
5
6  /**
7   * Cliente UDP: envía un datagrama y espera respuesta con
8   * timeout de 3 segundos.
9   */
10 public class ClienteUDP {
11
12     private static final String HOST    = "localhost";
13     private static final int    PUERTO  = 9001;
14     private static final int    TIMEOUT = 3000; // ms
15     private static final int    TAM_BUF = 1024;
16
17     public static void main(String[] args) throws Exception {
18         try (DatagramSocket socket = new DatagramSocket()) {
19             socket.setSoTimeout(TIMEOUT);
20             InetAddress servidor = InetAddress.getByName(HOST);
21             ;
22             Scanner sc = new Scanner(System.in);
23
24             System.out.println("Cliente UDP. Escribe un
25                               mensaje (o 'salir'):");
26             while (sc.hasNextLine()) {
27                 String linea = sc.nextLine().trim();
28                 if (linea.equalsIgnoreCase("salir")) break;
29
30                 byte[] datos = linea.getBytes();
31                 DatagramPacket pkt = new DatagramPacket(
32                     datos, datos.length, servidor, PUERTO);
33                 socket.send(pkt);
34
35                 // Esperar respuesta
36                 byte[] buf = new byte[TAM_BUF];
37                 DatagramPacket resp = new DatagramPacket(buf,
38                     buf.length);
39                 try {
40                     socket.receive(resp);

```

```

38         System.out.println("Respuesta: "
39             + new String(resp.getData(), 0, resp.
                getLength()));
40     } catch (SocketTimeoutException e) {
41         System.out.println("Timeout: no se recibio
                respuesta en "
42             + TIMEOUT + "ms");
43     }
44 }
45 }
46 }
47 }

```

Listing 1.5: ClienteUDP.java — Cliente sin conexión

1.5. Ejecución desde IntelliJ IDEA

Paso 1. Ejecutar **ServidorTCP**: clic derecho sobre la clase → *Run 'ServidorTCP.main()'*. Debe aparecer:

Resultado esperado en consola

```
[10:05:32.001] Servidor listo en :9000
```

Paso 2. Ejecutar **ClienteTCP**: ir a *Run* → *Edit Configurations* → *+* → *Application*, configurar la clase **ClienteTCP** y ejecutar. Probar los comandos:

Resultado esperado en consola

```

> HORA
Servidor: HORA_ACTUAL:10:05:40.234
> ECO Hola UTEQ
Servidor: ECO[#1]:Hola UTEQ
> SALIR
Servidor: ADIOS:hasta luego cliente #1

```

Paso 3. Para el UDP: ejecutar **ServidorUDP** primero, luego **ClienteUDP**. Para ejecutar múltiples instancias simultáneas del cliente, activar *Allow multiple instances* en la configuración de ejecución.

Nota importante

TCP usa `ServerSocket.accept()` que bloquea hasta recibir una conexión. Cada cliente se atiende en un hilo separado del `ExecutorService`. UDP usa `DatagramSocket.receive()` que bloquea hasta recibir un datagrama, sin estado entre paquetes.

Capítulo 2

Guía 2: WebSockets con Spring Boot

Objetivo: WebSockets con Spring Boot

Implementar un servidor de chat en tiempo real usando el protocolo WebSocket sobre HTTP/1.1, habilitando comunicación bidireccional y persistente entre clientes y servidor [fette2011].

2.1. Fundamento teórico

WebSocket (RFC 6455 [fette2011]) es un protocolo de comunicación que establece un canal full-duplex sobre una única conexión TCP. A diferencia de HTTP, que es petición-respuesta (half-duplex), WebSocket mantiene la conexión abierta permitiendo que el servidor envíe datos al cliente sin que este los solicite.

El ciclo de vida tiene tres fases:

1. **Handshake HTTP**: el cliente envía una cabecera `Upgrade: websocket`. El servidor responde `101 Switching Protocols`.
2. **Transferencia de frames**: datos se transmiten en frames binarios o de texto, eficientemente.
3. **Cierre**: cualquiera de los extremos envía un frame `CLOSE`.

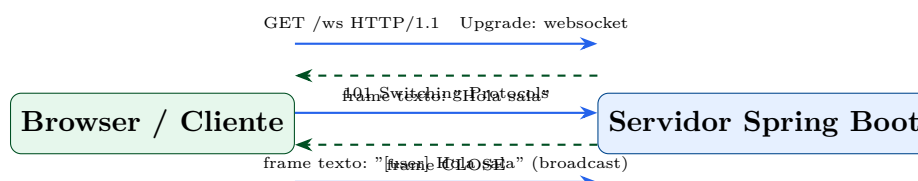


Figura 2.1: Ciclo de vida de una conexión WebSocket sobre HTTP

2.2. Crear proyecto Spring Boot

Paso 1. Abrir IntelliJ IDEA → *File* → *New* → *Project* → *Spring Initializr*. Configurar:

- **Name:** guia02-websocket
- **Language:** Java
- **Type:** Maven
- **Java:** 21
- **Package name:** ec.edu.uteq.distribuidas.ws

Paso 2. En la pantalla de dependencias, buscar y agregar: **WebSocket** (Spring WebSocket). Hacer clic en *Finish*.

El **pom.xml** generado incluirá automáticamente:

```

1      <dependency>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-starter-websocket</artifactId>
4      </dependency>
5      <dependency>
6      <groupId>org.webjars</groupId>
7      <artifactId>webjars-locator-core</artifactId>
8      </dependency>
```

Listing 2.1: Dependencia WebSocket en pom.xml

2.3. Configuración del servidor WebSocket

Paso 1. Crear la clase de configuración **WebSocketConfig.java**:

```

1      package ec.edu.uteq.distribuidas.ws.config;
2
3      import org.springframework.context.annotation.Configuration;
4      import org.springframework.messaging.simp.config.
5          MessageBrokerRegistry;
6      import org.springframework.web.socket.config.annotation.*;
7
8      /**
```

```

8      * Configura el broker de mensajes STOMP sobre WebSocket.
9      * STOMP (Simple Text Oriented Messaging Protocol) agrega
10     * un protocolo de mensajería sobre el canal WebSocket raw.
11     */
12     @Configuration
13     @EnableWebSocketMessageBroker
14     public class WebSocketConfig implements
15         WebSocketMessageBrokerConfigurer {
16
17         @Override
18         public void configureMessageBroker(MessageBrokerRegistry
19             config) {
20             // Prefijo de los topics a los que los clientes se
21             // suscriben
22             config.enableSimpleBroker("/topic");
23             // Prefijo de los mensajes enviados al servidor
24             config.setApplicationDestinationPrefixes("/app");
25         }
26
27         @Override
28         public void registerStompEndpoints(StompEndpointRegistry
29             registry) {
30             // Endpoint WebSocket con fallback SockJS para
31             // navegadores sin soporte
32             registry.addEndpoint("/ws-chat")
33                 .setAllowedOriginPatterns("*")
34                 .withSockJS();
35         }
36     }

```

Listing 2.2: WebSocketConfig.java — Configuración STOMP sobre WebSocket

Paso 2. Crear el modelo de mensaje **MensajeChat.java**:

```

1      package ec.edu.uteq.distribuidas.ws.model;
2
3      import java.time.LocalDateTime;
4      import java.time.format.DateTimeFormatter;
5
6      /**
7       * Record inmutable que representa un mensaje de chat.
8       * Los records de Java 16+ generan automáticamente constructor,
9       * getters, equals, hashCode y toString.
10     */
11     public record MensajeChat(
12         String usuario,

```

```

13     String contenido,
14     String tipo,        // "UNIRSE" | "MENSAJE" | "SALIR"
15     String timestamp
16 ) {
17     // Constructor compacto: valida y asigna timestamp si es
18     // null
19     public MensajeChat {
20         if (timestamp == null) {
21             timestamp = LocalDateTime.now()
22                 .format(DateTimeFormatter.ofPattern("HH:mm:ss"));
23         }
24     }
25 }

```

Listing 2.3: MensajeChat.java — Modelo de mensaje del chat

Paso 3. Crear el controlador de mensajes **ChatController.java**:

```

1     package ec.edu.uteq.distribuidas.ws.controller;
2
3     import ec.edu.uteq.distribuidas.ws.model.MensajeChat;
4     import org.springframework.messaging.handler.annotation.*;
5     import org.springframework.messaging.simp.
6         SimpMessageSendingOperations;
7     import org.springframework.stereotype.Controller;
8
9     /**
10      * Controlador de mensajes WebSocket.
11      * @MessageMapping mapea los mensajes STOMP enviados al
12      * servidor.
13      * @SendTo especifica el topic al que se reenvían los mensajes.
14      */
15     @Controller
16     public class ChatController {
17
18         private final SimpMessageSendingOperations
19             messagingTemplate;
20
21         public ChatController(SimpMessageSendingOperations
22             messagingTemplate) {
23             this.messagingTemplate = messagingTemplate;
24         }
25
26         /**
27          * Recibe mensajes en /app/chat.enviar
28          * y los reenvía a todos los suscriptores de /topic/sala-

```

```

25         general.
26         */
27         @MessagingMapping("/chat.enviar")
28         @SendTo("/topic/sala-general")
29         public MensajeChat enviarMensaje(MensajeChat mensaje) {
30             System.out.println("[WS] " + mensaje.usuario() + ": "
31                 + mensaje.contenido());
32             return mensaje;
33         }
34
35         /**
36         * Notifica a toda la sala cuando un usuario nuevo se une.
37         */
38         @MessagingMapping("/chat.unirse")
39         @SendTo("/topic/sala-general")
40         public MensajeChat unirse(MensajeChat mensaje,
41             @Header("simpSessionId") String sessionId) {
42             System.out.println("[WS] Usuario '" + mensaje.usuario
43                 ()
44                 + "' entró. SessionId: " + sessionId);
45             return new MensajeChat(mensaje.usuario(),
46                 mensaje.usuario() + " se unió a la sala.",
47                 "UNIRSE", null);
48         }
49     }
50 }

```

Listing 2.4: ChatController.java — Manejador de mensajes STOMP

Paso 4. Crear el cliente HTML **chat.html** en `src/main/resources/static` para probar desde el navegador:

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="UTF-8"/>
5  <title>Chat WebSocket - UTEQ</title>
6  <script src="https://cdn.jsdelivr.net/npm/sockjs-client@1/dist/
7      sockjs.min.js"></script>
8  <script src="https://cdn.jsdelivr.net/npm/stompjs@2.3.3/lib/stomp.
9      min.js"></script>
10 </head>
11 <body>
12 <h2>Chat en Tiempo Real (WebSocket + STOMP)</h2>
13 <input id="usuario" placeholder="Tu nombre" />
14 <button onclick="conectar()">Conectar</button>
15 <hr/>

```

```

14 <div id="mensajes" style="height:300px;overflow-y:auto;border:1px_
    solid_#ccc"></div>
15 <input id="texto" placeholder="Escribe_un_mensaje..." style="
    width:70%"/>
16 <button onclick="enviar()">Enviar</button>
17
18 <script>
19 let stompClient = null;
20
21 function conectar() {
22     const socket = new SockJS('/ws-chat');
23     stompClient = Stomp.over(socket);
24     stompClient.connect({}, frame => {
25         // Suscribirse al topic de la sala
26         stompClient.subscribe('/topic/sala-general', msg => {
27             const m = JSON.parse(msg.body);
28             mostrar('[' + m.timestamp + ']' + m.usuario + ': ' + m.
                contenido);
29         });
30         // Anunciar ingreso
31         stompClient.send('/app/chat.unirse', {},
32             JSON.stringify({usuario: document.getElementById('usuario').
                value,
33                 contenido: '', tipo: 'UNIRSE', timestamp: null}));
34     });
35 }
36
37 function enviar() {
38     const texto = document.getElementById('texto').value;
39     stompClient.send('/app/chat.enviar', {},
40         JSON.stringify({usuario: document.getElementById('usuario').
            value,
41             contenido: texto, tipo: 'MENSAJE', timestamp: null}));
42     document.getElementById('texto').value = '';
43 }
44
45 function mostrar(txt) {
46     const div = document.getElementById('mensajes');
47     div.innerHTML += '<p>' + txt + '</p>';
48     div.scrollTop = div.scrollHeight;
49 }
50 </script>
51 </body>
52 </html>

```

Listing 2.5: chat.html — Cliente WebSocket en el navegador

2.4. Ejecución

Paso 1. Ejecutar la clase principal de Spring Boot: clic derecho → *Run 'Guia02WebsocketApplication'*. Esperar hasta ver:

Resultado esperado en consola

```
Started Guia02WebsocketApplication in 2.3s (JVM running for 2.8)
Tomcat started on port 8080 (http)
```

Paso 2. Abrir **dos** pestañas de navegador en <http://localhost:8080/chat.html>. En cada pestaña introducir un nombre de usuario diferente, hacer clic en **Conectar** y enviar mensajes. Los mensajes deben aparecer en *ambas* pestañas en tiempo real.

Capítulo 3

Guía 3: RPC con Apache Thrift

Objetivo: RPC con Apache Thrift

Implementar un sistema de llamada a procedimiento remoto (RPC) usando Apache Thrift, que genera automáticamente el código de serialización (stub/skeleton) a partir de un archivo de definición de interfaz (`.thrift`) [slee2007].

3.1. Fundamento teórico

RPC (*Remote Procedure Call*) permite a un proceso invocar un procedimiento en otro proceso, posiblemente en otra máquina, como si fuera local. El mecanismo transparente de comunicación se logra mediante dos componentes generados automáticamente [birrell1984]:

- **Stub cliente** (proxy): intercepta la llamada, serializa los parámetros (*marshalling*), los envía por la red y devuelve el resultado.
- **Skeleton servidor**: recibe los bytes, los deserializa (*unmarshalling*) e invoca la implementación real.

Apache Thrift [slee2007] es un framework de Facebook (2007) que define interfaces en un IDL (*Interface Definition Language*) propio y genera código para más de 20 lenguajes.

3.2. Configuración del proyecto

Paso 1. Crear un proyecto Maven en IntelliJ con `GroupId ec.edu.uteq.distribuidas`, `ArtifactId guia03-rpc`.

Paso 2. Agregar la dependencia de Thrift en `pom.xml`:

```

1      <dependencies>
2      <dependency>
3      <groupId>org.apache.thrift</groupId>
4      <artifactId>libthrift</artifactId>
5      <version>0.19.0</version>
6      </dependency>
7      </dependencies>
8
9      <build>
10     <plugins>
11     <plugin>
12     <groupId>org.apache.thrift.tools</groupId>
13     <artifactId>maven-thrift-plugin</artifactId>
14     <version>0.1.11</version>
15     <configuration>
16     <thriftExecutable>thrift</thriftExecutable>
17     <generator>java</generator>
18     </configuration>
19     <executions>
20     <execution>
21     <id>thrift-sources</id>
22     <phase>generate-sources</phase>
23     <goals><goal>compile</goal></goals>
24     </execution>
25     </executions>
26     </plugin>
27     </plugins>
28     </build>

```

Listing 3.1: `pom.xml` — Dependencia Apache Thrift

Nota importante

Si Thrift no está instalado en el sistema, descargarlo de <https://thrift.apache.org/download> y agregar el ejecutable al PATH. En Ubuntu/Debian: `sudo apt install thrift-compiler`.

3.3. Definición del IDL

Paso 1. Crear el directorio `src/main/thrift` y dentro el archivo `calculadora.thrift`:

```

1  /**
2  * Definicion IDL de Thrift para una Calculadora Remota.
3  * El compilador de Thrift genera automaticamente:
4  *   - Calculadora.java (interfaz del servicio)
5  *   - Calculadora.Client (stub del cliente)
6  *   - Calculadora.Processor (skeleton del servidor)
7  */
8  namespace java ec.edu.uteq.distribuidas.rpc.gen
9
10 /** Excepcion de dominio para operaciones invalidas */
11 exception OperacionInvalidaException {
12     1: required string mensaje
13 }
14
15 /** Servicio de calculadora distribuida */
16 service Calculadora {
17
18     /** Suma dos numeros de doble precision */
19     double sumar(1: double a, 2: double b)
20
21     /** Resta b de a */
22     double restar(1: double a, 2: double b)
23
24     /** Multiplica dos numeros */
25     double multiplicar(1: double a, 2: double b)
26
27     /** Divide a entre b. Lanza excepcion si b == 0 */
28     double dividir(1: double a, 2: double b)
29     throws (1: OperacionInvalidaException ex)
30
31     /** Retorna el nombre del servidor que atendio */
32     string nombreServidor()
33 }

```

Listing 3.2: calculadora.thrift — Definición IDL de la calculadora remota

Paso 2. Generar el código Java. En la terminal integrada de IntelliJ (*View → Tool Windows → Terminal*):

```

# Desde la raiz del proyecto
thrift --gen java -out src/main/java src/main/thrift/calculadora.
thrift

```

Esto genera en `src/main/java/ec/edu/uteq/distribuidas/rpc/gen/` los archivos: `Calculadora.java` y `OperacionInvalidaException.java`.

3.4. Implementación servidor y cliente

Paso 1. Implementar la lógica del servidor en `CalculadoraImpl.java`:

```

1      package ec.edu.uteq.distribuidas.rpc.server;
2
3      import ec.edu.uteq.distribuidas.rpc.gen.Calculadora;
4      import ec.edu.uteq.distribuidas.rpc.gen.
5          OperacionInvalidaException;
6      import org.apache.thrift.TException;
7
8      /**
9       * Implementación concreta de la interfaz generada por Thrift.
10      * Esta es la logica de negocio real; el stub/skeleton
11      * son transparentes para el programador.
12      */
13      public class CalculadoraImpl implements Calculadora.Iface {
14
15          @Override
16          public double sumar(double a, double b) throws TException
17          {
18              System.out.printf("RPC sumar(%.2f, %.2f)%n", a, b);
19              return a + b;
20          }
21
22          @Override
23          public double restar(double a, double b) throws TException
24          {
25              System.out.printf("RPC restar(%.2f, %.2f)%n", a, b);
26              return a - b;
27          }
28
29          @Override
30          public double multiplicar(double a, double b) throws
31              TException {
32              System.out.printf("RPC multiplicar(%.2f, %.2f)%n", a,
33                  b);
34              return a * b;
35          }
36
37          @Override
38          public double dividir(double a, double b)
39              throws OperacionInvalidaException, TException {
40              System.out.printf("RPC dividir(%.2f, %.2f)%n", a, b);
41              if (b == 0.0) {
42                  throw new OperacionInvalidaException(

```

```

38         "Division por cero no permitida");
39     }
40     return a / b;
41 }
42
43 @Override
44 public String nombreServidor() throws TException {
45     return "ServidorThrift@" + java.net.InetAddress.
46         getLocalHost().getHostName();
47 }

```

Listing 3.3: CalculadoraImpl.java — Implementación de la lógica RPC

Paso 2. Crear **ServidorRPC.java** que arranca el servidor Thrift:

```

1  package ec.edu.uteq.distribuidas.rpc.server;
2
3  import ec.edu.uteq.distribuidas.rpc.gen.Calculadora;
4  import org.apache.thrift.server.*;
5  import org.apache.thrift.transport.*;
6
7  /**
8   * Servidor RPC usando Apache Thrift.
9   * TThreadPoolServer usa un pool de hilos para atender
10  * multiples clientes en paralelo.
11  */
12  public class ServidorRPC {
13
14      private static final int PUERTO = 9090;
15
16      public static void main(String[] args) throws Exception {
17          Calculadora.Processor<CalculadoraImpl> processor =
18              new Calculadora.Processor<>(new CalculadoraImpl());
19
20          TServerTransport transport = new TServerSocket(PUERTO)
21              ;
22
23          TThreadPoolServer.Args serverArgs =
24              new TThreadPoolServer.Args(transport)
25                  .processor(processor)
26                  .minWorkerThreads(5)
27                  .maxWorkerThreads(20);
28
29          TServer server = new TThreadPoolServer(serverArgs);
30          System.out.println("Servidor Thrift (RPC) escuchando

```

```

30         en : " + PUERTO);
        server.serve();    // BLOQUEA hasta que el servidor se
                           detenga
31     }
32 }

```

Listing 3.4: ServidorRPC.java — Servidor multihilo Thrift

Paso 3. Crear ClienteRPC.java:

```

1  package ec.edu.uteq.distribuidas.rpc.client;
2
3  import ec.edu.uteq.distribuidas.rpc.gen.*;
4  import org.apache.thrift.protocol.*;
5  import org.apache.thrift.transport.*;
6
7  /**
8   * Cliente que invoca metodos remotos en el servidor Thrift.
9   * Desde el codigo del cliente, llamar dividir() parece una
10  * llamada local, pero en realidad viaja por la red (RPC).
11  */
12  public class ClienteRPC {
13
14      private static final String HOST = "localhost";
15      private static final int PUERTO = 9090;
16
17      public static void main(String[] args) throws Exception {
18          try (TTransport transport = new TSocket(HOST, PUERTO))
19          {
20              transport.open();
21
22              TProtocol protocol = new TBinaryProtocol(transport);
23              Calculadora.Client cliente = new Calculadora.
24                  Client(protocol);
25
26              System.out.println("Conectado al servidor: " +
27                  cliente.nombreServidor());
28              System.out.printf("5 + 3 = %.2f\n", cliente.sumar
29                  (5, 3));
30              System.out.printf("10 - 4 = %.2f\n", cliente.
31                  restar(10, 4));
32              System.out.printf("6 x 7 = %.2f\n", cliente.
33                  multiplicar(6, 7));
34              System.out.printf("15 / 4 = %.4f\n", cliente.
35                  dividir(15, 4));

```

```
29
30      // Prueba de excepcion remota
31      try {
32          cliente.dividir(10, 0);
33      } catch (OperacionInvalidaException ex) {
34          System.out.println("Excepcion capturada: " +
35                              ex.getMessage());
36      }
37  }
38  }
```

Listing 3.5: ClienteRPC.java — Cliente RPC con Thrift

3.5. Ejecución

Paso 1. Ejecutar **ServidorRPC** primero. Salida esperada:

Resultado esperado en consola

```
Servidor Thrift (RPC) escuchando en :9090
```

Paso 2. Ejecutar **ClienteRPC**. Salida esperada:

Resultado esperado en consola

```
Conectado al servidor: ServidorThrift@mi-pc
5 + 3 = 8.00
10 - 4 = 6.00
6 x 7 = 42.00
15 / 4 = 3.7500
Excepcion capturada: Division por cero no permitida
```

Capítulo 4

Guía 4: gRPC con Protocol Buffers

Objetivo: gRPC con Protocol Buffers

Implementar un sistema de comunicación de alto rendimiento usando gRPC con Protocol Buffers (Protobuf) sobre HTTP/2, incluyendo llamadas unarias y streaming bidireccional [indrasiri2020].

4.1. Fundamento teórico

gRPC (Google Remote Procedure Call) es un framework RPC moderno que usa **HTTP/2** como transporte y **Protocol Buffers** (Protobuf) como formato de serialización binario [indrasiri2020].

Ventajas sobre alternativas:

- **HTTP/2**: multiplexing de múltiples RPCs sobre una sola conexión TCP (sin head-of-line blocking), compresión de headers (HPACK).
- **Protobuf**: 3-10× más compacto que JSON; 5-10× más rápido de serializar/deserializar.
- **Tipado fuerte**: el compilador `protoc` genera código verificado en tiempo de compilación para más de 20 lenguajes.
- **Streaming bidireccional**: cliente y servidor pueden enviarse flujos de mensajes simultáneamente.

gRPC define cuatro tipos de llamadas:

1. **Unaria**: cliente envía un mensaje, recibe uno.
2. **Server streaming**: cliente envía uno, recibe un stream.
3. **Client streaming**: cliente envía stream, recibe uno.

4. **Bidireccional:** ambos envían y reciben streams simultáneamente.

4.2. Configuración del proyecto

Paso 1. Crear proyecto Maven en IntelliJ con ArtifactId guia04-grpc. Configurar **pom.xml** con las dependencias de gRPC y el plugin protoc:

```

1      <properties>
2      <maven.compiler.source>21</maven.compiler.source>
3      <maven.compiler.target>21</maven.compiler.target>
4      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
5      <grpc.version>1.62.2</grpc.version>
6      <protobuf.version>3.25.3</protobuf.version>
7      <protoc.version>3.25.3</protoc.version>
8      </properties>
9
10     <dependencies>
11     <dependency>
12     <groupId>io.grpc</groupId>
13     <artifactId>grpc-netty-shaded</artifactId>
14     <version>${grpc.version}</version>
15     </dependency>
16     <dependency>
17     <groupId>io.grpc</groupId>
18     <artifactId>grpc-protobuf</artifactId>
19     <version>${grpc.version}</version>
20     </dependency>
21     <dependency>
22     <groupId>io.grpc</groupId>
23     <artifactId>grpc-stub</artifactId>
24     <version>${grpc.version}</version>
25     </dependency>
26     <dependency>
27     <groupId>com.google.protobuf</groupId>
28     <artifactId>protobuf-java</artifactId>
29     <version>${protobuf.version}</version>
30     </dependency>
31     <!-- Anotacion @Generated requerida por el codigo generado -->
32     <dependency>
33     <groupId>javax.annotation</groupId>
34     <artifactId>javax.annotation-api</artifactId>
35     <version>1.3.2</version>
36     </dependency>
37     </dependencies>
38

```

```

39     <build>
40     <extensions>
41     <extension>
42     <groupId>kr.motd.maven</groupId>
43     <artifactId>os-maven-plugin</artifactId>
44     <version>1.7.1</version>
45     </extension>
46     </extensions>
47     <plugins>
48     <plugin>
49     <groupId>org.xolstice.maven.plugins</groupId>
50     <artifactId>protobuf-maven-plugin</artifactId>
51     <version>0.6.1</version>
52     <configuration>
53     <protocArtifact>
54     com.google.protobuf:protoc:${protoc.version}:exe:${os.detected.
55         classifier}
56     </protocArtifact>
57     <pluginId>grpc-java</pluginId>
58     <pluginArtifact>
59     io.grpc:protoc-gen-grpc-java:${grpc.version}:exe:${os.detected.
60         classifier}
61     </pluginArtifact>
62     </configuration>
63     <executions>
64     <execution>
65     <goals>
66     <goal>compile</goal>
67     <goal>compile-custom</goal>
68     </goals>
69     </execution>
70     </executions>
71     </plugin>
72     </plugins>
73     </build>

```

Listing 4.1: pom.xml — Dependencias gRPC con generación automática

4.3. Definición del contrato Protobuf

Paso 1. Crear el directorio `src/main/proto` y el archivo `mensajeria.proto`:

```

1     syntax = "proto3";
2
3     package ec.edu.uteq.distribuidas.grpc;

```

```

4   option java_package = "ec.edu.uteq.distribuidas.grpc.gen";
5   option java_outer_classname = "MensajeriaProto";
6   option java_multiple_files = true;
7
8   // -- Mensajes de solicitud y respuesta -----
9   message SolicitudMensaje {
10      string remitente    = 1;
11      string contenido     = 2;
12      int64  timestamp    = 3;  // Reloj de Lamport
13  }
14
15  message RespuestaMensaje {
16      string servidor      = 1;
17      string contenido     = 2;
18      int64  timestamp    = 3;
19      bool   exito        = 4;
20  }
21
22  message SolicitudStream {
23      string usuario = 1;
24      int32  cantidad = 2;  // Cuantos mensajes transmitir
25  }
26
27  // -- Definicion del servicio gRPC -----
28  service ServicioMensajeria {
29
30      // Llamada unaria: cliente envia 1 mensaje, recibe 1 respuesta
31      rpc EnviarMensaje(SolicitudMensaje) returns (RespuestaMensaje);
32
33      // Server streaming: el servidor envia multiples respuestas
34      rpc RecibirNoticias(SolicitudStream) returns (stream
35          RespuestaMensaje);
36
37      // Bidireccional: cliente y servidor intercambian streams
38      rpc ChatBidireccional(stream SolicitudMensaje)
39          returns (stream RespuestaMensaje);
40  }

```

Listing 4.2: mensajeria.proto — Definición del servicio gRPC

Paso 2. Generar el código Java ejecutando desde la terminal de IntelliJ:

```
mvn generate-sources
```

El código se genera en `target/generated-sources/protobuf/`. IntelliJ lo detecta automáticamente si se ejecutó *Maven* → *Reload Project*.

4.4. Implementación del servidor gRPC

Paso 1. Crear `MensajeríaServiceImpl.java`:

```

1      package ec.edu.uteq.distribuidas.grpc.server;
2
3      import ec.edu.uteq.distribuidas.grpc.gen.*;
4      import io.grpc.stub.StreamObserver;
5
6      import java.util.concurrent.atomic.AtomicLong;
7
8      /**
9       * Implementacion de los tres tipos de llamada gRPC:
10      *   1. Unaria
11      *   2. Server streaming
12      *   3. Bidireccional
13      */
14     public class MensajeríaServiceImpl
15     extends ServicioMensajeríaGrpc.ServicioMensajeríaImplBase {
16
17         private final AtomicLong relojLamport = new AtomicLong(0);
18
19         // -- 1. Llamada unaria -----
20         @Override
21         public void enviarMensaje(SolicitudMensaje request,
22         StreamObserver<RespuestaMensaje> responseObserver) {
23             // Regla R3 del reloj de Lamport
24             long ts = Math.max(relojLamport.get(), request.
25                 getTimestamp()) + 1;
26             relojLamport.set(ts);
27
28             System.out.printf("[gRPC Unario] %s: %s (ts=%d)%n",
29                 request.getRemitente(), request.getContenido(), ts);
30
31             RespuestaMensaje respuesta = RespuestaMensaje.
32                 newBuilder()
33                 .setServidor("ServidorGRPC")
34                 .setContenido("ACK: " + request.getContenido())
35                 .setTimestamp(ts)
36                 .setExito(true)
37                 .build();
38
39             responseObserver.onNext(respuesta);
40             responseObserver.onCompleted();
41         }
42     }

```

```

41 // -- 2. Server streaming -----
42 @Override
43 public void recibirNoticias(SolicitudStream request,
44 StreamObserver<RespuestaMensaje> responseObserver) {
45     String usuario = request.getUsuario();
46     int cantidad = request.getCantidad();
47
48     System.out.printf("[gRPC Streaming] Enviando %d
49         noticias a %s\n",
50         cantidad, usuario);
51
52     for (int i = 1; i <= cantidad; i++) {
53         long ts = relojLamport.incrementAndGet();
54         RespuestaMensaje noticia = RespuestaMensaje.
55             newBuilder()
56             .setServidor("ServidorGRPC")
57             .setContenido("Noticia #" + i + " para " + usuario
58             )
59             .setTimestamp(ts)
60             .setExito(true)
61             .build();
62         responseObserver.onNext(noticia);
63
64         try { Thread.sleep(500); } catch (
65             InterruptedException ignored) {}
66     }
67     responseObserver.onCompleted();
68 }
69
70 // -- 3. Chat bidireccional -----
71 @Override
72 public StreamObserver<SolicitudMensaje> chatBidireccional(
73 StreamObserver<RespuestaMensaje> responseObserver) {
74
75     return new StreamObserver<>() {
76         @Override
77         public void onNext(SolicitudMensaje msg) {
78             long ts = Math.max(relojLamport.get(), msg.
79                 getTimestamp()) + 1;
80             relojLamport.set(ts);
81             System.out.printf("[gRPC Bidir] %s: %s\n",
82                 msg.getRemitente(), msg.getContenido());
83             // Eco con el timestamp actualizado
84             responseObserver.onNext(RespuestaMensaje.
85                 newBuilder()
86                 .setServidor("ServidorGRPC")
87                 .setContenido "[" + msg.getRemitente() + "]" "

```

```

82         + msg.getContenido())
83         .setTimestamp(ts)
84         .setExito(true)
85         .build());
86     }
87
88     @Override
89     public void onError(Throwable t) {
90         System.err.println("Error en stream: " + t.
91             getMessage());
92     }
93
94     @Override
95     public void onCompleted() {
96         responseObserver.onCompleted();
97     }
98 }
99

```

Listing 4.3: MensajeriaServiceImpl.java — Implementación de los tres tipos de llamada

Paso 2. Crear **ServidorGRPC.java**:

```

1     package ec.edu.uteq.distribuidas.grpc.server;
2
3     import io.grpc.Server;
4     import io.grpc.ServerBuilder;
5
6     public class ServidorGRPC {
7
8         private static final int PUERTO = 50051;
9
10        public static void main(String[] args) throws Exception {
11            Server server = ServerBuilder.forPort(PUERTO)
12                .addService(new MensajeriaServiceImpl())
13                .build()
14                .start();
15
16            System.out.println("Servidor gRPC iniciado en puerto "
17                + PUERTO);
18            Runtime.getRuntime().addShutdownHook(new Thread(server
19                ::shutdown));
20            server.awaitTermination();
21        }
22    }
23

```

Listing 4.4: ServidorGRPC.java — Servidor gRPC sobre HTTP/2

Paso 3. Crear **ClienteGRPC.java** para probar los tres tipos de llamada:

```

1      package ec.edu.uteq.distribuidas.grpc.client;
2
3      import ec.edu.uteq.distribuidas.grpc.gen.*;
4      import io.grpc.*;
5      import io.grpc.stub.StreamObserver;
6
7      import java.util.Iterator;
8      import java.util.concurrent.*;
9      import java.util.concurrent.atomic.AtomicLong;
10
11     public class ClienteGRPC {
12
13         private static final AtomicLong reloj = new AtomicLong(0);
14
15         public static void main(String[] args) throws Exception {
16             ManagedChannel canal = ManagedChannelBuilder
17                 .forAddress("localhost", 50051)
18                 .usePlaintext()           // Sin TLS (solo para
19                     desarrollo)
20                 .build();
21
22             ServicioMensajeriaGrpc.ServicioMensajeriaBlockingStub
23             stubBloqueante = ServicioMensajeriaGrpc.
24                 newBlockingStub(canal);
25             ServicioMensajeriaGrpc.ServicioMensajeriaStub
26             stubAsync = ServicioMensajeriaGrpc.newStub(canal);
27
28             // -- 1. Llamada unaria -----
29             System.out.println("=== 1. Llamada Unaria ===");
30             SolicitudMensaje solicitud = SolicitudMensaje.
31                 newBuilder()
32                 .setRemitente("ClienteA")
33                 .setContenido("Hola desde gRPC")
34                 .setTimestamp(reloj.incrementAndGet())
35                 .build();
36             RespuestaMensaje resp = stubBloqueante.enviarMensaje(
37                 solicitud);
38             System.out.println("Respuesta: " + resp.getContenido()
39                 + " (ts=" + resp.getTimestamp() + ")");
40         }
41     }

```

```

37 // -- 2. Server streaming -----
38 System.out.println("\n== 2. Server Streaming (3
    noticias) ==");
39 Iterator<RespuestaMensaje> noticias = stubBloqueante.
    recibirNoticias(
40     SolicitudStream.newBuilder()
41     .setUsuario("ClienteA")
42     .setCantidad(3)
43     .build());
44 while (noticias.hasNext()) {
45     System.out.println(" << " + noticias.next().
        getContenido());
46 }
47
48 // -- 3. Bidireccional -----
49 System.out.println("\n== 3. Chat Bidireccional ==");
50 CountdownLatch latch = new CountdownLatch(1);
51 StreamObserver<SolicitudMensaje> emisor =
52 stubAsync.chatBidireccional(new StreamObserver<>() {
53     @Override
54     public void onNext(RespuestaMensaje r) {
55         System.out.println(" << " + r.getContenido());
56         ;
57     }
58     @Override
59     public void onError(Throwable t) {
60         latch.countDown();
61     }
62     @Override
63     public void onCompleted() {
64         System.out.println("Stream completado.");
65         latch.countDown();
66     }
67 });
68
69 // Enviar 3 mensajes en el stream del cliente
70 for (int i = 1; i <= 3; i++) {
71     emisor.onNext(SolicitudMensaje.newBuilder()
72         .setRemitente("ClienteA")
73         .setContenido("Mensaje bidireccional #" + i)
74         .setTimestamp(reloj.incrementAndGet())
75         .build());
76     Thread.sleep(300);
77 }
78 emisor.onCompleted();
79 latch.await(5, TimeUnit.SECONDS);

```

```
80         canal.shutdown();  
81     }  
82 }
```

Listing 4.5: ClienteGRPC.java — Cliente que demuestra los tres tipos de llamada

4.5. Ejecución

Paso 1. Ejecutar *Maven* → *generate-sources* para compilar el `.proto`. Luego ejecutar **ServidorGRPC**. Verificar:

Resultado esperado en consola

```
Servidor gRPC iniciado en puerto 50051
```

Paso 2. Ejecutar **ClienteGRPC**. Salida esperada:

Resultado esperado en consola

```
=== 1. Llamada Unaria ===  
Respuesta: ACK: Hola desde gRPC (ts=2)  
  
=== 2. Server Streaming (3 noticias) ===  
<< Noticia #1 para ClienteA  
<< Noticia #2 para ClienteA  
<< Noticia #3 para ClienteA  
  
=== 3. Chat Bidireccional ===  
<< [ClienteA] Mensaje bidireccional #1  
<< [ClienteA] Mensaje bidireccional #2  
<< [ClienteA] Mensaje bidireccional #3  
Stream completado.
```

Capítulo 5

Guía 5: Java RMI (Remote Method Invocation)

Objetivo: Java RMI

Implementar un sistema RMI nativo de Java que permita invocar métodos de objetos remotos como si fueran locales, usando el registro RMI como servicio de nombres y la serialización de objetos Java [waldo1994].

5.1. Fundamento teórico

RMI (Remote Method Invocation) es la implementación de RPC de Java, introducida en Java 1.1. Permite a un programa Java invocar métodos de un objeto Java ubicado en otra JVM (Java Virtual Machine), posiblemente en otro host [waldo1994].

Arquitectura de tres capas:

- **Capa de stub/skeleton:** generados en tiempo de compilación (Java 5+: generados automáticamente, no requiere `rmic`).
- **Capa de referencia remota:** maneja la semántica de llamada (at-most-once), gestión de la conexión TCP subyacente.
- **Capa de transporte:** usa JRMP (*Java Remote Method Protocol*) sobre TCP. Serialización nativa de Java.

El **RMI Registry** (puerto 1099 por defecto) actúa como servicio de nombres: el servidor registra sus objetos con `Naming.bind()`, el cliente los localiza con `Naming.lookup()` [vanSteen2023].

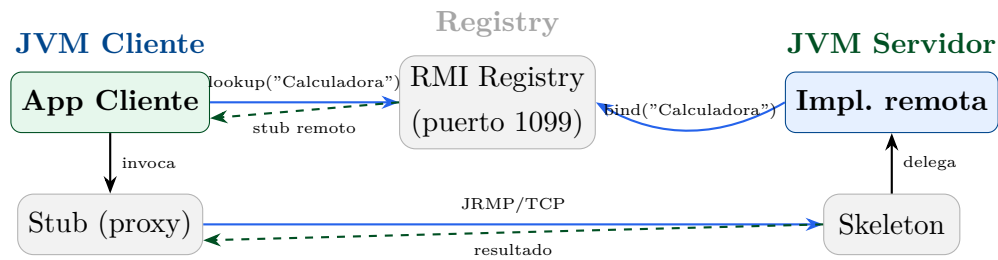


Figura 5.1: Arquitectura completa del sistema RMI con tres JVMs

5.2. Estructura del proyecto

Paso 1. Crear proyecto Maven con ArtifactId `guia05-rmi`. No se requieren dependencias externas: RMI es parte del JDK estándar.

```

1  <properties>
2  <maven.compiler.source>21</maven.compiler.source>
3  <maven.compiler.target>21</maven.compiler.target>
4  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
5  </properties>
6  <!-- Sin dependencias adicionales: RMI esta en java.rmi del JDK --
   >
    
```

Listing 5.1: pom.xml para proyecto RMI

5.3. Interfaz remota

Paso 1. Crear el paquete `ec.edu.uteq.distribuidas.rmi.api` y la interfaz remota `CalculadoraRemota.java`:

```

1  package ec.edu.uteq.distribuidas.rmi.api;
2
3  import java.rmi.Remote;
4  import java.rmi.RemoteException;
5
6  /**
7   * Interfaz remota RMI.
8   * REGLAS OBLIGATORIAS para interfaces RMI:
9   * 1. Debe extender java.rmi.Remote.
10  * 2. Cada metodo DEBE declarar throws RemoteException
11  *    (para manejar fallos de red transparentemente).
12  * 3. Los parametros y el valor de retorno deben ser:
13  *    - Tipos primitivos, o
    
```

```

14      *      - Implementar java.io.Serializable, o
15      *      - Implementar Remote (objetos remotos anidados).
16      */
17      public interface CalculadoraRemota extends Remote {
18
19          double sumar(double a, double b) throws RemoteException;
20          double restar(double a, double b) throws RemoteException;
21          double multiplicar(double a, double b) throws
22              RemoteException;
23          double dividir(double a, double b) throws RemoteException;
24          String obtenerInfo() throws RemoteException;
25
26          // Metodos adicionales (Java 21)
27          double potencia(double base, double exponente) throws
28              RemoteException;
29          double raizCuadrada(double n) throws RemoteException;
30      }

```

Listing 5.2: CalculadoraRemota.java — Interfaz del objeto remoto

5.4. Implementación del objeto remoto

Paso 1. Crear el paquete `ec.edu.uteq.distribuidas.rmi.server` y la implementación `CalculadoraImpl.java`:

```

1      package ec.edu.uteq.distribuidas.rmi.server;
2
3      import ec.edu.uteq.distribuidas.rmi.api.CalculadoraRemota;
4      import java.rmi.RemoteException;
5      import java.rmi.server.UnicastRemoteObject;
6
7      /**
8       * Implementacion concreta del objeto remoto.
9       *
10      * OBLIGATORIO: extender UnicastRemoteObject.
11      * Esto hace que el objeto:
12      * - Se exporte automaticamente al instanciarse.
13      * - Use TCP para recibir llamadas remotas.
14      * - Sea elegible para garbage collection remoto.
15      *
16      * Alternativa: NO extender UnicastRemoteObject y usar
17      *   UnicastRemoteObject.exportObject(this, 0) manualmente.
18      */
19      public class CalculadoraImpl

```

```
20 extends UnicastRemoteObject
21 implements CalculadoraRemota {
22
23     private static final long serialVersionUID = 1L;
24
25     private final String nombreServidor;
26
27     // El constructor DEBE declarar throws RemoteException
28     public CalculadoraImpl(String nombre) throws
29         RemoteException {
30         super(); // exporta el objeto (asigna puerto TCP
31         aleatorio)
32         this.nombreServidor = nombre;
33     }
34
35     @Override
36     public double sumar(double a, double b) throws
37         RemoteException {
38         logLlamada("sumar", a, b);
39         return a + b;
40     }
41
42     @Override
43     public double restar(double a, double b) throws
44         RemoteException {
45         logLlamada("restar", a, b);
46         return a - b;
47     }
48
49     @Override
50     public double multiplicar(double a, double b) throws
51         RemoteException {
52         logLlamada("multiplicar", a, b);
53         return a * b;
54     }
55
56     @Override
57     public double dividir(double a, double b) throws
58         RemoteException {
59         logLlamada("dividir", a, b);
60         if (b == 0.0) {
61             // Se puede lanzar RemoteException o una excepcion
62             serializable propia
63             throw new RemoteException("Division por cero no
64                 permitida");
65         }
66         return a / b;
67     }
68 }
```

```

59     }
60
61     @Override
62     public double potencia(double base, double exp) throws
        RemoteException {
63         return Math.pow(base, exp);
64     }
65
66     @Override
67     public double raizCuadrada(double n) throws
        RemoteException {
68         if (n < 0) throw new RemoteException("Raiz de negativo
            no definida en R");
69         return Math.sqrt(n);
70     }
71
72     @Override
73     public String obtenerInfo() throws RemoteException {
74         return "Servidor RMI: " + nombreServidor
75             + " | JVM: " + System.getProperty("java.version")
76             + " | Hilos activos: " + Thread.activeCount();
77     }
78
79     private void logLlamada(String metodo, double a, double b)
80     {
81         System.out.printf("[RMI] %s(%.2f, %.2f) invocado%n",
            metodo, a, b);
82     }

```

Listing 5.3: CalculadoraImpl.java — Implementación del objeto remoto

Paso 2. Crear **ServidorRMI.java** que arranca el registry y exporta el objeto:

```

1     package ec.edu.uteq.distribuidas.rmi.server;
2
3     import java.rmi.Naming;
4     import java.rmi.registry.LocateRegistry;
5
6     /**
7     * Servidor RMI:
8     * 1. Crea el RMI Registry en el puerto 1099.
9     * 2. Instancia el objeto remoto (lo exporta automaticamente)
10    *
11    * 3. Lo registra en el Registry con un nombre.
12    */

```

```
12     public class ServidorRMI {
13
14         private static final int PUERTO_REGISTRY = 1099;
15         private static final String NOMBRE      = "
16             CalculadoraDistribuida";
17
18         public static void main(String[] args) {
19             try {
20                 // Paso 1: Crear el registry en este proceso
21                 LocateRegistry.createRegistry(PUERTO_REGISTRY);
22                 System.out.println("RMI Registry creado en puerto
23                     " + PUERTO_REGISTRY);
24
25                 // Paso 2: Instanciar e implementacion (se exporta
26                     automaticamente)
27                 CalculadoraImpl calculadora = new CalculadoraImpl(
28                     "UTEQ-Servidor-01");
29
30                 // Paso 3: Registrar con el nombre buscable por
31                     clientes
32                 // URL: rmi://host:puerto/nombre
33                 String url = "rmi://localhost:" + PUERTO_REGISTRY
34                     + "/" + NOMBRE;
35                 Naming.bind(url, calculadora);
36
37                 System.out.println("Objeto registrado en: " + url)
38                     ;
39                 System.out.println("Servidor RMI listo. Esperando
40                     clientes...");
41                 System.out.println(calculadora.obtenerInfo());
42
43                 // El servidor se mantiene vivo mientras el
44                     registry exista
45                 // En produccion usar join() en lugar de sleep
46                 Thread.currentThread().join();
47
48             } catch (Exception e) {
49                 System.err.println("Error en ServidorRMI: " + e.
50                     getMessage());
51                 e.printStackTrace();
52             }
53         }
54     }
```

Listing 5.4: ServidorRMI.java — Servidor que registra el objeto remoto

5.5. Cliente RMI

Paso 1. Crear el paquete `ec.edu.uteq.distribuidas.rmi.client` y la clase `ClienteRMI.java`:

```

1      package ec.edu.uteq.distribuidas.rmi.client;
2
3      import ec.edu.uteq.distribuidas.rmi.api.CalculadoraRemota;
4      import java.rmi.Naming;
5      import java.rmi.RemoteException;
6
7      /**
8       * Cliente RMI.
9       * IMPORTANTE: El cliente SOLO necesita la interfaz
10        CalculadoraRemota.java.
11        * NO necesita la implementacion CalculadoraImpl.java.
12        * Esta es la "transparencia de acceso" de RMI.
13        */
14      public class ClienteRMI {
15
16          private static final String HOST    = "localhost";
17          private static final int    PUERTO  = 1099;
18          private static final String NOMBRE  = "
19              CalculadoraDistribuida";
20
21          public static void main(String[] args) {
22              String url = "rmi://" + HOST + ":" + PUERTO + "/" +
23                  NOMBRE;
24
25              try {
26                  // lookup() devuelve un stub (proxy) que parece un
27                  // objeto local
28                  CalculadoraRemota calc =
29                      (CalculadoraRemota) Naming.lookup(url);
30
31                  System.out.println("Conectado a: " + calc.
32                      obtenerInfo());
33                  System.out.println("
34                      -----");
35
36                  // Todas estas llamadas viajan por la red
37                  // transparentemente
38                  System.out.printf("10 + 5    = %.2f%n", calc.sumar
39                      (10, 5));
40                  System.out.printf("10 - 5    = %.2f%n", calc.restar
41                      (10, 5));

```

```
33      System.out.printf("10 x 5    = %.2f%n", calc.  
34          multiplicar(10, 5));  
35      System.out.printf("10 / 4    = %.4f%n", calc.  
36          dividir(10, 4));  
37      System.out.printf("2 ^ 10    = %.0f%n", calc.  
38          potencia(2, 10));  
39      System.out.printf("sqrt(2)   = %.6f%n", calc.  
40          raizCuadrada(2));  
41  
42      // Prueba de excepcion remota  
43      try {  
44          calc.dividir(5, 0);  
45      } catch (RemoteException e) {  
46          System.out.println("Excepcion remota: " + e.  
47              getCause().getMessage());  
48      }  
49  
50      } catch (Exception e) {  
51          System.err.println("Error en ClienteRMI: " + e.  
52              getMessage());  
53      }  
54  }  
55  }
```

Listing 5.5: ClienteRMI.java — Cliente que invoca métodos remotos

5.6. Ejecución

Paso 1. Ejecutar **ServidorRMI** primero. Salida esperada:

Resultado esperado en consola

```
RMI Registry creado en puerto 1099  
Objeto registrado en: rmi://localhost:1099/  
    CalculadoraDistribuida  
Servidor RMI listo. Esperando clientes...  
Servidor RMI: UTEQ-Servidor-01 | JVM: 21.0.3 | Hilos activos  
    : 4
```

Paso 2. Ejecutar **ClienteRMI**. Salida esperada:

Resultado esperado en consola

```
Conectado a: Servidor RMI: UTEQ-Servidor-01 | JVM: 21.0.3 |
Hilos activos: 5
-----
10 + 5    = 15.00
10 - 5    = 5.00
10 x 5    = 50.00
10 / 4    = 2.5000
2 ^ 10    = 1024
sqrt(2)   = 1.414214
Excepcion remota: Division por cero no permitida
```

Advertencia

Si aparece `java.rmi.ConnectException: Connection refused to host`: verificar que el **ServidorRMI** esté ejecutándose antes que el cliente. En redes locales con múltiples interfaces de red, puede ser necesario agregar `-Djava.rmi.server.hostname=IP_del_servidor` como argumento de JVM al servidor.

Paso 3. Para simular dos JVMs en equipos distintos: cambiar **HOST** en **ClienteRMI.java** a la IP del servidor. El cliente solo necesita la interfaz **CalculadoraRemota.java** en su classpath.

Capítulo 6

Análisis Comparativo

6.1. Cuadro sinóptico de mecanismos

Cuadro 6.1: Comparativa técnica de los cinco mecanismos de comunicación implementados

Criterio	TC- P/UDP	WebSoc- ket	Thrift RPC	gRPC	RMI
Protocolo base	TCP / UDP	HTTP → WS	TCP (bin)	HTTP/2	JRM- P/TCP
Serialización	Manual	JSON/texto	Thrift bin	Protobuf	Java ser.
Tipado IDL	Ninguno	Ninguno	.thrift	.proto	Java iface
Streaming	Sí (TCP)	Bidireccional	No nativo	Sí (4 tipos)	No
Multilenguaje	Sí	Sí	20+ lang.	20+ lang.	Solo Java
Overhead (aprox.)	Muy bajo	Bajo	Bajo	Muy bajo	Alto
Complejidad conf.	Baja	Media	Media	Media	Baja
Caso de uso ideal	Chat, bin	Tiempo real	Microserv.	Microserv.	Java-Java

6.2. Criterios de selección

Para elegir el mecanismo de comunicación en un proyecto distribuido, considerar:

1. **¿Es solo Java?** RMI es la opción más simple. De lo contrario, descartar RMI.
2. **¿Requiere tiempo real en el browser?** WebSocket es la única opción con soporte nativo en navegadores sin polling.
3. **¿Rendimiento crítico entre microservicios?** gRPC con Protobuf ofrece la mejor relación latencia/throughput.

4. **£Control total sobre la red?** Sockets TCP/UDP permiten optimizar cada byte, al costo de mayor complejidad de desarrollo.
5. **£Necesita interoperar con sistemas legados?** Apache Thrift tiene soporte para más lenguajes incluyendo C++ y Erlang.